Control Structures and microprogramming

Subject : Computer Architecture

Control Structure

- A control structure in computer architecture is the logical organization and design of the CPU that determines how instructions are executed.
- It defines the sequence of operations, the generation of control signals, and the path of data and signals between different components to ensure correct instruction execution.

- Control Structure = The CPU’s “plan or flow” that manages how, when, and where each operation happens during instruction execution.

Components of Control Structure

1.Instruction Cycle

- In order for a single instruction to be executed by the CPU, it must go through the instruction cycle (also sometimes referred to as the fetch-execute cycle). While this cycle can vary from CPU to CPU, they typically consist of the following stages:
- Fetch
- Decode
- Execute
- Memory Access
- Registry Write-Back

Components of Control Structure

- **Fetch:**

To execute a single instruction, the CPU must first fetch it from memory. The Program Counter (PC) stores the address of the next instruction, which is then copied into the Instruction Register (IR). This allows the CPU to access the instruction and prepare it for processing. Think of it like giving your order at a deli, where a ticket is created for the staff to process when your turn comes.

- **Decode:**

After fetching, the CPU decodes the instruction to understand what operation needs to be performed. The Control Unit (CU) converts the instruction into control signals that tell different parts of the CPU what to do. Similar to the deli example, this is like the staff reading your ticket to figure out exactly what you ordered.

Components of Control Structure

- **Execute:**

During execution, the CPU performs the operation specified by the instruction. The control signals guide the ALU or other components to carry out the task. In the deli analogy, this stage is when the staff actually makes your sandwich according to the order.

- **Memory Access:**

Some instructions require extra data from memory. If so, the CPU retrieves the needed information before completing the instruction. For example, if a calculation depends on previously stored values, the CPU accesses those memory addresses. At the deli, this is like fetching an ingredient needed for your order before making the sandwich.

Components of Control Structure

- **Register Write-Back:**

Finally, if the instruction changes any data, the result is written back into the CPU registers. This step ensures that new or updated data is stored for future instructions. In the deli example, it’s like putting your sandwich on the “specials board” to keep a record of the completed order.

Components of Control Structure

2.Timing and Control Signals:

Timing and control signals are the commands generated by the CPU’s control unit to manage the execution of instructions. They determine when each component of the CPU is activated, ensuring the proper sequence of operations. For example, signals control when registers load data, which operation the ALU performs, and when memory read/write operations occur. These signals synchronize all CPU components so that no part operates too early or too late, maintaining proper execution timing.

Components of Control Structure

3.Control Paths / Signal Flow:

Control paths define how control signals and data flow through CPU components. They guide instructions from the Program Counter (PC) to the Memory Address Register (MAR), then to Memory, Instruction Register (IR), ALU, and back to registers. This structured flow ensures that instructions are fetched, decoded, and executed in the correct sequence. Without a proper control path, the CPU cannot maintain order or accuracy in instruction execution.

Components of Control Structure

4.Branching and Decision Making:

Control structures also handle branching and decision making in instruction execution. The CPU can perform conditional or unconditional branches depending on the status of flags or status bits in the registers. This allows the CPU to choose the next instruction to execute based on certain conditions, enabling loops, decisions, and subroutine calls in programs.

Branching means **changing the program counter (PC)** so that execution jumps to a different memory address.

Types of Branching

1.Unconditional Branch

1. Always jumps to a new address.
2. Example: JMP LABEL

2.Conditional Branch

1. Jumps only if a condition is true (based on flags).
2. Example: JZ, JNZ, JC, JNC, JG, JL

3.Call / Return Branch

1. Jumps to a subroutine and comes back.
2. Example: CALL, RET

Types of Control Structure:

- **Hardwired Control Structure:** This type uses fixed logic circuits to generate control signals. It is fast because signals are directly generated by hardware but is less flexible, meaning changes require hardware modification.

- **Microprogrammed Control Structure:** This type stores control sequences as microinstructions in control memory. It is more flexible, allowing easy updates or new instructions, but slightly slower than hardwired control.

Operation / How It Works Hardwired Control Structure

When the CPU fetches an instruction, its opcode is loaded into the Instruction Register (IR). This opcode is decoded by an Instruction Decoder, which activates specific outputs corresponding to that instruction. These outputs feed into combinational logic and sequence generators which, together with timing signals and possibly status/flag registers, generate the required control signals (for example: incrementing Program Counter, loading Memory Address Register, enabling ALU operations, memory read/write). The control logic thus transitions through a sequence of states (a finite-state machine) for each instruction, based on clocks, interrupts or condition flags, and the wiring determines the next state and the outputs.

Characteristics / Features Hardwired Control Structure

- Generates control signals directly via hardware circuits → very fast
- Low flexibility: hardware redesign required for instruction set changes
- Best suited for simple/fixed instruction sets (common in RISC processors)

Advantages

- Very fast signal generation, because control logic is built from hardware gates and circuits with minimal latency.
- No need for a separate control memory (as used in micro-programmed units), reducing overhead in that regard.
- Efficient for simple instruction sets where change is unlikely and high performance is desired.

Disadvantages

- Poor flexibility: changes to the instruction set, or modifications to control flow, require redesigning the control logic circuit.
- Complexity and cost increase significantly when instructions grow in number or complexity, since the wiring and logic becomes more elaborate.
- Difficult to debug, maintain, or extend; the physical nature of the circuits makes updates challenging.

DATA FLOW IN MICROPROCESSING (Datapath Flow)

Data flow in a microprocessor describes **how data moves inside the CPU** during instruction execution.

1. Registers

•Store data temporarily

•Examples: ACC, PC, IR, MAR, MDR, general-purpose registers (R1, R2…)

2. ALU (Arithmetic Logic Unit)

•Performs operations like add, subtract, AND, OR, shift, compare

3. Internal Buses

•**Data Bus** → carries data

•**Address Bus** → carries addresses

•**Control Bus** → carries control signals

4. Memory

•Stores instructions & data

5. Control Unit

•Controls data movement using control signals

Advantages of Microprocessors

1. High Speed

Microprocessors can execute millions of instructions per second, making systems very fast.

2. Small Size

They integrate thousands/millions of transistors in a small chip, reducing system size.

3. Low Cost

Mass production makes microprocessors affordable compared to mechanical or older electronic systems.

4. Low Power Consumption

Most microprocessors use small power and support power-saving modes.

5. Versatility / Programmability

A single microprocessor can be programmed to perform multiple tasks, from calculators to computers.

Disadvantages of Microprocessors

1. Limited to Binary Operations

Microprocessors work with binary logic; they cannot handle analog data directly (need ADC/DAC).

2. Heating Problem

High-speed processors produce heat and require cooling systems.

3. No Internal Memory

Basic microprocessors (like 8085/8086) do not have internal RAM/ROM; external memory is required.

4. Requires Skilled Programmers

Writing efficient assembly/machine code needs expertise.

5. Limited Bandwidth of Buses

Data flow bottlenecks can slow performance (Von Neumann bottleneck).

Usage & Implementation

Hardwired control units are commonly used in processors where performance is critical and the instruction set is relatively stable and simple such as many RISC-based processors. In contrast, processors with large or evolving instruction sets (for example many CISC designs) often prefer microprogrammed control units to allow easier updates and modifications of control logic.